# Learning When to Think: RL for Adaptive Reasoning Computation in LLMs

**Ivan Andhika**
School of Computing
University of Utah
u1590903@utah.edu

**Hao Ren**
School of Computing
University of Utah
u1527543@utah.edu

**Ammon Gleason**
School of Computing
University of Utah
u1221580@utah.edu

## Abstract

Large language models (LLMs) benefit from additional test-time computation, but fixed reasoning budgets are inefficient: easy problems are over-processed while hard problems still fail. We study adaptive reasoning as a reinforcement learning problem where the model decides when to continue, refine, or terminate. Our implementation uses a three-action policy and Group Relative Policy Optimization with two key modifications: Adaptive Length Penalty (ALP), which scales token cost by per-prompt group solve rate, and a DeGRPO-style objective that up-weights control-token gradients relative to response-token gradients. We train Qwen2.5-Math-7B-Instruct with LoRA over attention and MLP modules and evaluate in a MATH-500 setting. Results are reported with accuracy and efficiency metrics (tokens, cost per correct), plus action usage analysis to verify adaptive compute allocation behavior.

## 1 Introduction

Increasing test-time computation improves LLM reasoning performance, but naive scaling is compute-inefficient [Snell et al., 2024]. Chain-of-thought and self-consistency methods [Wei et al., 2022, Wang et al., 2023] apply mostly fixed budgets and do not explicitly learn when to stop or when to self-correct. In practice, this causes two failure modes: overthinking on easy instances and insufficient exploration on difficult instances.

We ask whether an LLM can learn to allocate reasoning effort per instance. We cast reasoning as a sequential decision process with three actions: **continue**, **refine**, and **terminate**. The refine action is an explicit self-correction mode: the model revisits recent reasoning before proceeding, inspired by process supervision and self-correction paradigms [Lightman et al., 2023, Kumar et al., 2024, Uesato et al., 2022]. This design targets a realistic setting where control quality (which action to choose) matters as much as response quality (what tokens to generate).

Our implementation aligns policy, objective, and evaluation: (1) ALP reward shaping based on per-prompt group solve rate; (2) DeGRPO-style weighted gradients separating control tokens from response tokens; and (3) a MATH-500 benchmark setup with competition-style answer extraction (`\boxed{}` and `####`). The overall goal is to improve the accuracy-efficiency frontier rather than maximizing accuracy alone.

## 2 Related Work

**Chain-of-thought and self-consistency.** Prior work shows that intermediate reasoning and multi-sample voting improve math performance [Wei et al., 2022, Wang et al., 2023], but usually with

Class Project for Advanced AI, Spring 2025.

fixed decode budgets and no learned stopping policy. Our method keeps these as baselines and optimizes a policy that can vary compute by instance.

**RL for LLM reasoning.** GRPO-style methods optimize relative advantages within rollout groups without a critic network, making them practical for LLM fine-tuning [Shao et al., 2024]. We build on this paradigm and adapt it to control-sensitive optimization where mode-selection tokens are explicitly emphasized.

**Adaptive compute and length control.** Adaptive length penalties use online difficulty estimates (e.g., solve-rate signals) to avoid over-penalizing long reasoning on hard inputs [Snell et al., 2024]. We adopt this principle through ALP, where the token penalty scales with per-prompt group solve rate.

**Self-correction as a policy action.** Process-level supervision and self-correction literature motivates explicit correction steps [Lightman et al., 2023, Zhang et al., 2025, Kumar et al., 2024]. In our framework, `refine` is not a passive formatting token but a policy decision that triggers targeted re-checking before further generation.

# 3 Method

## 3.1 Problem Formulation

We model reasoning as an MDP. Given problem $q$, the state $s_t$ contains the prompt and all prior reasoning turns. The action space is

$$\mathcal{A} = \{\texttt{continue}, \texttt{refine}, \texttt{terminate}\}.$$

**Continue** extends the current reasoning trajectory. **Refine** applies a self-correction nudge that asks the model to re-check and fix recent reasoning before proceeding. **Terminate** ends the episode and returns the final answer. Episodes stop at `terminate` or at a maximum step budget $T$.

To support ablations, we include an `allow_refine` switch that disables `refine` and reduces the action set to continue/terminate.

## 3.2 Reward Function

We use Adaptive Length Penalty (ALP) to balance correctness and efficiency:

$$R_k = r_{\text{acc},k} - \beta \cdot \max(0, \text{SR}(q)) \cdot \frac{n_{\text{tokens},k}}{L_{\max}} \tag{1}$$

where $k$ indexes rollouts for the same prompt, $r_{\text{acc},k} \in \{0, 1\}$ is exact-match correctness, and $\text{SR}(q) = \frac{1}{K}\sum_k r_{\text{acc},k}$ is group solve rate (an online proxy for prompt difficulty). $n_{\text{tokens},k}$ is rollout length and $L_{\max}$ is a normalization cap (configured directly or derived from decode limits). This formulation penalizes length more strongly when a prompt is already easy for the current policy (high solve rate), and less strongly when the prompt is hard (low solve rate).

## 3.3 Training Approaches

**Primary training method (GRPO + DeGRPO weighting):** For each prompt, we sample $K$ rollouts, compute ALP rewards, normalize advantages within the group, and optimize policy log-probabilities. Following the audit-guided design, we split generated tokens per step into control and response slices and use weighted loss

$$\mathcal{L} = -A_k \left(w_{\text{ctrl}} \log p_{\text{ctrl}} + w_{\text{resp}} \log p_{\text{resp}}\right) + \lambda_{\text{KL}}\text{KL},$$

with $w_{\text{ctrl}} > w_{\text{resp}}$ to prevent control-signal washout. A `degrpo=false` flag restores vanilla uniform weighting for ablation.

**LoRA adaptation:** We use LoRA on both attention and MLP projections (e.g., `q/k/v/o_proj`, `gate/up/down_proj`), with configurable rank and alpha.

**Implementation scope:** The current paper template describes the active MATH-500 pipeline. Other training routes (e.g., SFT/DPO) are optional and not required for the core experiments reported here.

# 4 Experimental Design

## 4.1 Hypotheses

1. **H1:** ALP + DeGRPO improves the accuracy–efficiency Pareto frontier over fixed-compute and vanilla-GRPO baselines on MATH-500.
2. **H2:** The learned policy allocates more computation (tokens/steps) to harder prompts and less to easier prompts.
3. **H3:** Action usage shifts with difficulty, with relatively more `refine`/`continue` on difficult cases and more `terminate` on easier cases.

## 4.2 Domain, Setup, and Evaluation

We use **MATH-500** for the main experiments, with Qwen2.5-Math-7B-Instruct as base model and LoRA adaptation (default $r$=16, $\alpha$=32). Outputs are graded using normalized math-string matching from final `####` and/or `\boxed{...}` extraction. We treat GSM8K [Cobbe et al., 2021] as optional sanity-check context, not as the primary benchmark in this template.

**Baselines:** (1) CoT single-pass decoding; (2) direct-answer baseline; (3) vanilla GRPO objective (`degrpo=false`); (4) no-refine policy (`allow_refine=false`) for action-space ablation.

**Our method:** GRPO with ALP reward and DeGRPO weighted control-vs-response optimization.

**Metrics:** accuracy, average tokens per problem, cost per correct answer, average steps, parse/format success rates, and aggregate action counts. We report comparisons on fixed MATH-500 subsets configured in YAML. Ablations over $\beta$, $L_{\max}$, `degrpo` on/off, and control weighting (`w_ctrl`, `w_resp`, `n_control_tokens`) characterize the accuracy–efficiency trade-off.

# References


Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.

Aviral Kumar, Vincent Zhuang, Rishabh Agarwal, Yi Su, John D Co-Reyes, et al. Training language models to self-correct via reinforcement learning. *arXiv preprint arXiv:2409.12917*, 2024.

Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let's verify step by step. *arXiv preprint arXiv:2305.20050*, 2023.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y.K. Li, Y. Wu, and Daya Guo. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

Charlie Snell, Jaeho Lee, Kelvin Xu, and Aviral Kumar. Scaling llm test-time compute optimally can be more effective than scaling model parameters. *arXiv preprint arXiv:2408.03314*, 2024.

Jonathan Uesato, Nate Kushman, Ramana Kumar, Francis Song, Noah Siegel, Lisa Wang, Antonia Creswell, Geoffrey Irving, Irina Higgins, Dorsa Sadigh, et al. Solving math word problems with process- and outcome-based feedback. *arXiv preprint arXiv:2211.14275*, 2022.

Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V. Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In *International Conference on Learning Representations (ICLR)*, 2023. URL `https://arxiv.org/abs/2203.11171`.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, pages 24824–24837, 2022.

Zhenru Zhang, Chujie Zheng, Yangzhen Wu, Beichen Zhang, Runji Lin, Bowen Yu, Dayiheng Liu, Jingren Zhou, and Junyang Lin. The lessons of developing process reward models in mathematical reasoning. *arXiv preprint arXiv:2501.07301*, 2025.